

Connecting to GitHub

Test your SSH connection

- **First, check if your Git SSH key is working:**
 - Open your terminal (or Git Bash on Windows) and run the command:
`ssh -T git@github.com`
 - You should see a welcome message containing your username

```
(base) karunyashirali@Sophia-MBPro-438349 ShadowSimulation % ssh -T git@github.com  
Hi KarunyaShirali! You've successfully authenticated, but GitHub does not provide shell access.
```

- If it doesn't work, you will need to troubleshoot (verify your key/add a new key)
- Next, activate your virtual environment: `conda activate molssi_best_practices`

Adding your local project to GitHub

- **Create a Repository on GitHub:**
 - Sign in to your GitHub account on your browser.
 - Click the + icon in the top-right and select **New repository**.
 - Name your repository (the same as your local project) and click **Create repository**. Do **not** initialize it with a README, .gitignore, or license.
- Note: I've set my visibility to Public here (you can set it to Private if you'd like), but you'll need to make it Public when submitting your assignment

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner *

 KarunyaShirali

Repository name *

monte_carlo

✔ monte_carlo is available.

Great repository names are short and memorable. How about [bookish-lamp?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

Adding your local project to GitHub

- **Push your Repository to GitHub from your local machine:**
 - Open a terminal window (either in VSCode or the native terminal on your laptop).
 - Navigate to **your project folder** (montecarlo).
 - Run:

```
git init
git add .
git commit -m "Initial commit"
```
 - Run the following code, replacing the GitHub URL in it with the URL for your newly created repository

```
git remote add origin https://github.com/KarunyaShirali/monte_carlo.git
git branch -M main
git push -u origin main
```



Next step: Documentation

- Documentation helps others understand your project
- Makes installation and usage easy
- Improves collaboration, essential for open-source projects

- How do we go about setting it up?
- Let's use the Sphinx package
- **Activate your virtual environment:** `conda activate molssi_best_practices`
- **Install Sphinx:** `conda install sphinx sphinx-rtd-theme`

Build the Documentation locally

- Navigate to the docs/ directory in your package in the terminal
- Run
`sphinx-quickstart`
in the terminal in the docs/ directory to generate the configuration and Makefile
- It will ask you a couple of prompts
- You can answer “n” to the question “Separate source and build directories”
- Build the documentation: Run
`make html`
from the docs/ directory.
- The output will be in docs/_build/html
- You can try opening any of the html files in your browser to view them

Use index.md as the documentation source

- Run `conda install -c conda-forge myst-parser`
- Remove the `index.rst` file: `rm index.rst`
- In the `conf.py` file in `docs/`, make the following changes:

```
extensions = ['myst_parser',]
```

```
source_suffix = {
```

```
    '.rst': 'restructuredtext',
```

```
    '.md': 'markdown',
```

```
}
```

```
# Set the master toctree document to index.md
```

```
master_doc = 'index'
```

- Run `make html` from the `docs/` folder

Maintaining your docs

- Cookiecutter creates a set of doc files for you (those referenced in index.md)
- You can add to these
- Add docstrings: for example

```
def hello(name: str) -> str:
    """Return a greeting.
    Args:
        name (str): Person's name
    Returns:
        str: Greeting message
    """
    return f"Hello, {name}!"
```

- Keep it updated:
 - Ensure examples still run
 - Keep instructions simple and clear